

EDA Script

2026-06-08

```
# Global options for clean executive PDF execution
knitr::opts_chunk$set(
  echo = TRUE,
  warning = FALSE,
  message = FALSE,
  fig.width = 7,
  fig.height = 5,
  fig.align = "center"
)
```

```
# Load required packages
library(tidyverse)
# Data manipulation & snake_case conversion via janitor library(janitor) \# For automated vari

library(janitor)      # For automated variable cleaning (snake_case)
library(Amelia)       # Missing data visualizations
library(naniar)       # Advanced missingness plots
library(mice)         # Multivariate Imputation by Chained Equations
library(psych)        # Bartlett's test and KMO index
library(visdat)       # Visual snapshot
```

```
#Introduction/Data Ingestion
```

```
# Ingestion: Pull directly from a reliable public mirror of the IBM Attrition dataset
url <- "https://raw.githubusercontent.com/pplonski/datasets-for-start/refs/heads/master/employ
```

```
#Create a data frame for imported data
```

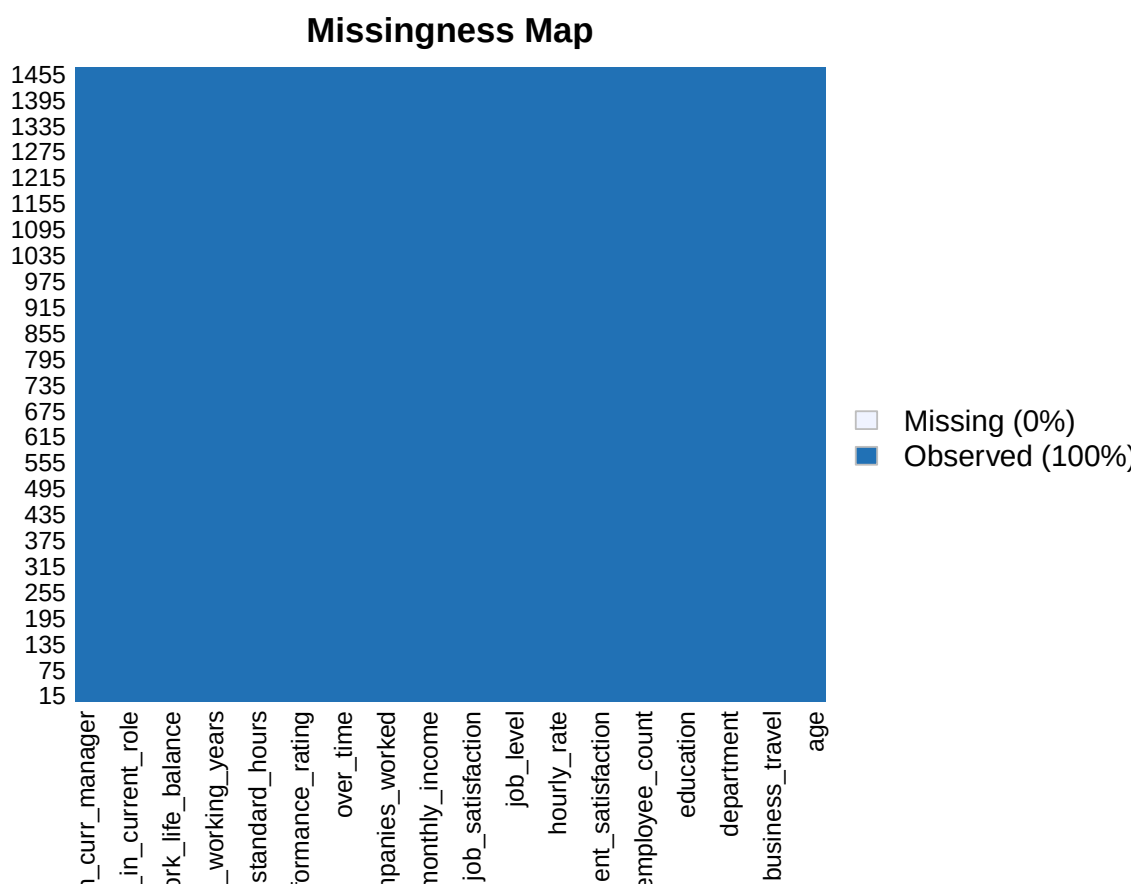
```
raw_df <- read_csv(url)
```

Convert all variables to clean snake_case to match production guidelines

Review structure of the first few columns

#Missing Data

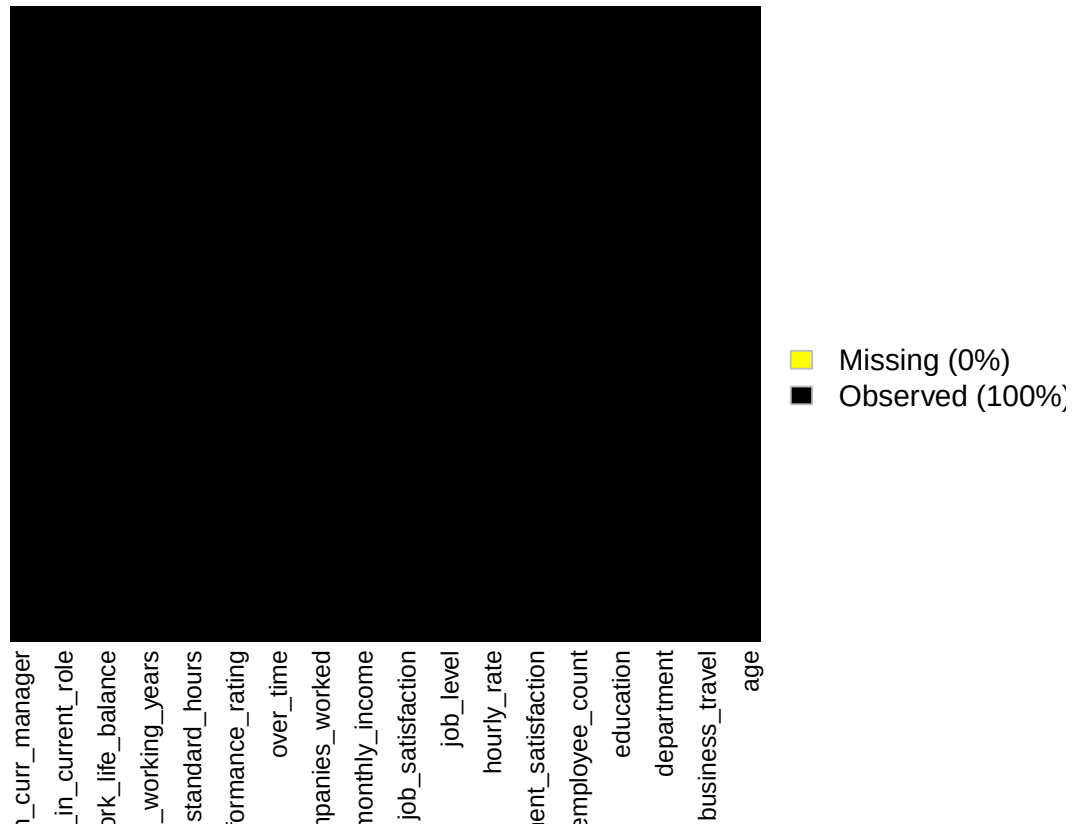
```
#' Looking for missing data  
misssmap(df)
```



Visually show missing data

```
misssmap(df, y.at=c(1), y.labels=c(''), col=c('yellow', 'black'))
```

Missingness Map



Show percent of missing data

```
percentmissing = function (x){ sum(is.na(x))/length(x) * 100}

missing <- apply(df, 1, percentmissing) # we will use an apply function to loop it. 1 indicates

table(round(missing, 1))
```

```
0
1470
```

#Remove anybody missing more than 5% of the data

```
replace_people <- subset(df, missing <= 5)
```

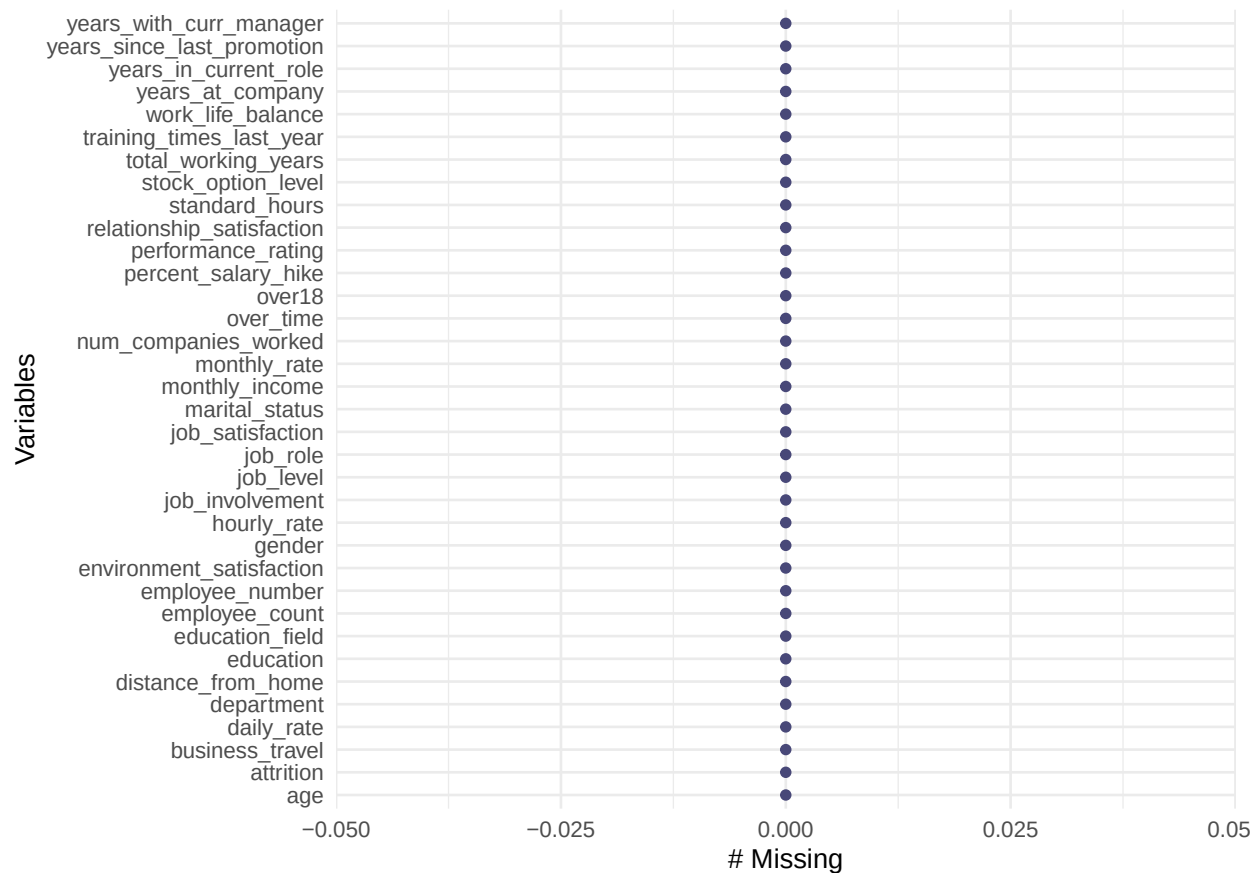
Displaying first 10 columns for cleaner PDF rendering spacing

```
apply(replace_people, 2, percentmissing)[1:10] #notice the 2 for column!
```

age	attrition	business_travel	daily_rate
0	0	0	0
department	distance_from_home	education	education_field
0	0	0	0
employee_count	employee_number		
0	0		

Using naniar to look at missing variables visually

```
replace_people %>%  
  gg_miss_var()
```



```
#Imputing Data Process
```

```
set.seed(2026) #for reproducibility
```

We run MICE on our dataset. It automatically bypasses columns without missingness.

```
temp_nomiss <- mice(replace_people, m = 1, method = "pmm", printFlag = FALSE)
nomiss <- complete(temp_nomiss, 1)
```

Confirm summary of the dataset post-imputation evaluation

Length is sample size

```
summary(nomiss[, 1:5])
```

age	attrition	business_travel	daily_rate
Min. :18.00	Length:1470	Length:1470	Min. : 102.0
1st Qu.:30.00	Class :character	Class :character	1st Qu.: 465.0
Median :36.00	Mode :character	Mode :character	Median : 802.0
Mean :36.92			Mean : 802.5
3rd Qu.:43.00			3rd Qu.:1157.0
Max. :60.00			Max. :1499.0
department			
Length:1470			
Class :character			
Mode :character			

```
#Outlier Detection
```

```
nomiss_numeric <- nomiss %>%
  select(where(is.numeric)) %>%
  select(-any_of(c("employee_count", "standard_hours", "employee_number")))
```

```
# Set the random seed for reproducibility
set.seed(2026)
```

```
# Compute the chi-square cutoff for identifying multivariate outliers.
# p = .999 means we're using the 99.9th percentile as our threshold (very conservative).
# df = number of variables in the dataset, because Mahalanobis distance follows a
# chi-square distribution with degrees of freedom equal to the number of variables.
cutoff <- qchisq(
  p = .999,
  df = ncol(nomiss_numeric)
)
```

Calculate the Mahalanobis distance for each observation in the dataset.

This measures how many multivariate standard deviations each case is from

the centroid (multivariate mean), accounting for correlations among variables.

```
mahal = mahalanobis(nomiss_numeric,
                    colMeans(nomiss_numeric),
                    cov(nomiss_numeric))
```

Print the cutoff value so you can inspect the threshold being applied

```
cutoff ##cutoff score
```

```
[1] 49.72823
```

Print the degrees of freedom (number of variables) used in the chi-square distribution

```
ncol(nomiss_numeric) ##df
```

```
[1] 23
```

```
#Items identified as false are outliers
```

```
# Evaluate which cases fall BELOW the cutoff (i.e., are NOT outliers).  
# TRUE  = within acceptable distance from centroid (keep)  
# FALSE = exceeds cutoff, flagged as a multivariate outlier (review/remove)  
table(mahal < cutoff)
```

```
FALSE  TRUE  
18    1452
```

```
“{r setup, include=FALSE} # Global options for clean executive PDF execution knitr::opts_chunk$set(  
echo = TRUE, warning = FALSE, message = FALSE, fig.width = 7, fig.height = 5, fig.align =  
“center” )
```

```
#Items identified as outliers
```

```
# Use bind_cols to cleanly attach the calculated distances vector  
nomiss_mahal <- nomiss_numeric %>%  
  bind_cols(mahal = mahal)
```

flagged rows that exceed our conservative cutoff and identified as outliers

```
mahal_out <- nomiss_mahal %>%  
  filter(mahal > cutoff) %>%  
  arrange(desc(mahal)) # sort mahal values from most to least  
  
# Previewing high-distance multivariate combinations  
head(mahal_out[, c("age", "monthly_income", "total_working_years", "mahal")])
```

	age	monthly_income	total_working_years	mahal
1	58	10312	40	99.95082
2	55	13577	34	90.41411
3	58	13872	38	76.61553
4	55	19586	36	74.79919
5	46	17567	27	73.46432
6	52	16856	34	72.23764

```
#Final outlier omission
```

```
##exclude outliers  
noout <- nomiss %>%  
  filter(mahal < cutoff)
```

##additivity We extract the numeric attributes from the final outlier-omitted dataset. #We are looking for any 1s off the diagonal, which would represent perfect #multi-collinearity or redundant data tracking.

```
final_numeric <- noout %>%
  select(where(is.numeric)) %>%
  select(-any_of(c("employee_count", "standard_hours", "employee_number")))

correl = cor(final_numeric, use = "pairwise.complete.obs")

# Print symbolic matrix check
symnum(correl[, 1:12])
```

```

              a dl_ d__ ed e_ h jb_n jb_l jb_s mnthly_n mnthly_r n
age              1
daily_rate        1
distance_from_home      1
education              1
environment_satisfaction      1
hourly_rate          1
job_involvement        1
job_level            .              1
job_satisfaction              1
monthly_income        .              *              1
monthly_rate              .              1
num_companies_worked    .              1
percent_salary_hike
performance_rating
relationship_satisfaction
stock_option_level
total_working_years    ,              ,              ,
training_times_last_year
work_life_balance
years_at_company              .              .
years_in_current_role              .              .
years_since_last_promotion              .              .
years_with_curr_manager              .              .
attr(,"legend")
[1] 0 ' ' 0.3 '.' 0.6 ',' 0.8 '+' 0.9 '*' 0.95 'B' 1

```

correlation adequacy Bartlett's test

```
# Run Bartlett's Test of Sphericity on the correlation matrix.
# H0: The correlation matrix is an identity matrix (all off-diagonal r = 0)
# H1: The correlation matrix is NOT an identity matrix (meaningful correlations exist)
cortest.bartlett(correl, n = nrow(noout))
```



```
$chisq
[1] 12010.17
```

```
$p.value
[1] 0
```

```
$df
[1] 253
```

sampling adequacy KMO test

```
# Run the Kaiser-Meyer-Olkin (KMO) Measure of Sampling Adequacy.
# We pass our complete numeric correlation matrix.
KMO(correl)
```

Kaiser-Meyer-Olkin factor adequacy

Call: KMO(r = correl)

Overall MSA = 0.78

MSA for each item =

age	daily_rate
0.81	0.43
distance_from_home	education
0.34	0.76
environment_satisfaction	hourly_rate
0.45	0.50
job_involvement	job_level
0.47	0.75
job_satisfaction	monthly_income
0.47	0.74
monthly_rate	num_companies_worked
0.57	0.74
percent_salary_hike	performance_rating
0.50	0.50
relationship_satisfaction	stock_option_level
0.49	0.52
total_working_years	training_times_last_year
0.87	0.53
work_life_balance	years_at_company
0.36	0.82
years_in_current_role	years_since_last_promotion
0.88	0.93
years_with_curr_manager	
0.87	

#Alternative Drop Processing Pipeline “{r}”

```
#' Let's drop the missing data for now # df_dropped <- na.omit(df) #' Check for missing data  
again # missmap(df_dropped, y.at=c(1), y.labels=c(' '), col=c('yellow', 'black'))
```